This will result in the predicate being checked more often than it might otherwise be, but it's easily used in place of a plain call to `wait()`. The variants with timeouts are easily implemented: wait either for the time specified, or 1 ms, whichever is shortest. OK, so `std::condition_variable` waits are now taken care of; what about `std::condition_variable_any`? Is this the same, or can you do better?

### 9.2.4. Interrupting a wait on std::condition_variable_any

`std::condition_variable_any` differs from `std::condition_variable` in that it works with *any* lock type rather than just `std::unique_lock<std::mutex>`. It turns out that this makes things much easier, and you *can* do better with `std::condition_variable_any` than you could with `std::condition_variable`. Because it works with *any* lock type, you can build your own lock type that locks/unlocks both the internal `set_clear_mutex` in your `interrupt_flag` *and* the lock supplied to the wait call, as shown here.

**Listing 9.12. `interruptible_wait` for `std::condition_variable_any`**